

Software Architecture and Design Specification

Project: Collaborative Study Material & Quiz Platform (CSQP)

Version: 1.0 | Date: 04-10-2026 | Status: Final

Authors: Harshith R Reddy (PES2UG24CS188), Kritika Mahapatra (PES2UG24CS239), Krishna Manoj (PES2UG24CS236), John S Thomas (PES2UG24CS207)

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Team 4	Final SADS document prepared for project evaluation

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Team Member Contribution

No.	Team Member	Assigned Work
1	Harshith R Reddy (PES2UG24CS188)	Introduction, Document Overview, Component Architecture Diagram & Descriptions
2	Kritika Mahapatra (PES2UG24CS239)	Architectural Patterns, Stakeholders & Concerns, UML Sequence Diagrams
3	Krishna Manoj (PES2UG24CS236)	REST API Design, Database Schemas, Error Handling, Logging & UX Design
4	John S Thomas (PES2UG24CS207)	Security Architecture, STRIDE Threat Modeling, Requirements Traceability Matrix

Table of Contents

1. Introduction	- 2
1.1 Purpose	- 2
1.2 Scope	- 2
1.3 Audience	- 2
1.4 Definitions	- 2
2. Document Overview	- 2
2.1 How to use this document	- 2
2.2 Related Documents	- 2
3. Architecture	- 3
3.1 Goals & Constraints	- 3
3.2 Stakeholders & Concerns	- 3
3.3 Component (UML) Diagram	- 3
3.4 Component Descriptions	- 4
3.5 Chosen Architecture Pattern and Rationale	- 4
3.6 Technology Stack & Data Stores	- 4
3.7 Risks & Mitigations	- 4
3.8 Traceability to Requirements	- 5
3.9 Security Architecture (STRIDE Threat Model)	- 5
4. Design	- 6
4.1 Design Overview	- 6
4.2 UML Sequence Diagrams (Diagram 1 & 2)	- 6
4.3 API Design	- 8
4.4 Error Handling, Logging & Monitoring	- 8
4.5 UX Design	- 9
4.6 Open Issues & Next Steps	- 9
5. Appendices	- 9
5.1 Glossary	- 9
5.2 References	- 9
5.3 Tools	- 9

1. Introduction

1.1 Purpose: This document specifies the software architecture and system design of the Collaborative Study Material & Quiz Platform (CSQP). It defines the subsystem decomposition, design decisions, data models, communication interfaces, and runtime interactions to fulfill requirements specified in the SRS v1.0.

1.2 Scope: Covers client-server interactions for student note uploads, shared question bank curation, instructor moderation, anti-cheat timed quiz taking, and analytics reporting. Encompasses the React frontend, Node.js/Express API, MongoDB persistence layer, and Python analytical routines. Excludes external campus LMS software.

1.3 Audience: Developers (Team 4), QA engineers, course evaluators, instructors, and system administrators.

1.4 Definitions: MERN (MongoDB, Express, React, Node.js), RBAC (Role-Based Access Control), JWT (JSON Web Token), MCQ (Multiple Choice Question), STRIDE (Security Threat Model), DTO (Data Transfer Object), TLS (Transport Layer Security), RTM (Requirements Traceability Matrix).

2. Document Overview

2.1 How to use this document: Serves as the technical reference for implementing, verifying, and assessing the platform. Section 3 covers system-level architecture, component topologies, and threat models. Section 4 specifies detailed behavioral sequence flows, REST API contracts, and error handling. Section 5 provides reference materials and tool definitions.

2.2 Related Documents: Software Requirements Specification (SRS v1.0), Software Test Plan (STP v1.0), and Requirements Traceability Matrix (RTM).

3. Architecture

3.1 Goals & Constraints:

- **Goals:** High responsiveness ($p_{90} \leq 2s$ response time for page loads and submissions), 99.5% monthly availability, robust role isolation (Student, Instructor, Admin), and quiz academic integrity.
- **Constraints:** Implementation within a single academic semester by a 4-member student team using JavaScript/Python (MERN stack). Hosted under free/low-cost cloud tiers with standard modern browser compatibility.

3.2 Stakeholders & Concerns:

- **Students:** Intuitive note discovery, prompt feedback on quiz attempts, and assurance of fair anti-cheat quiz conditions.
- **Instructors:** Streamlined question approval queues, flexible quiz authoring (manual & sampled), and clear class-level performance tracking.
- **Administrators:** Secure role assignment, non-repudiation audit logging, and platform operational health.
- **Evaluators:** Clean IEEE standard compliance, maintainable separation of concerns, and full requirement traceability.

3.3 Component (UML) Diagram:

Figure 3.1 visualizes the primary subsystems, internal modules, and data boundaries:

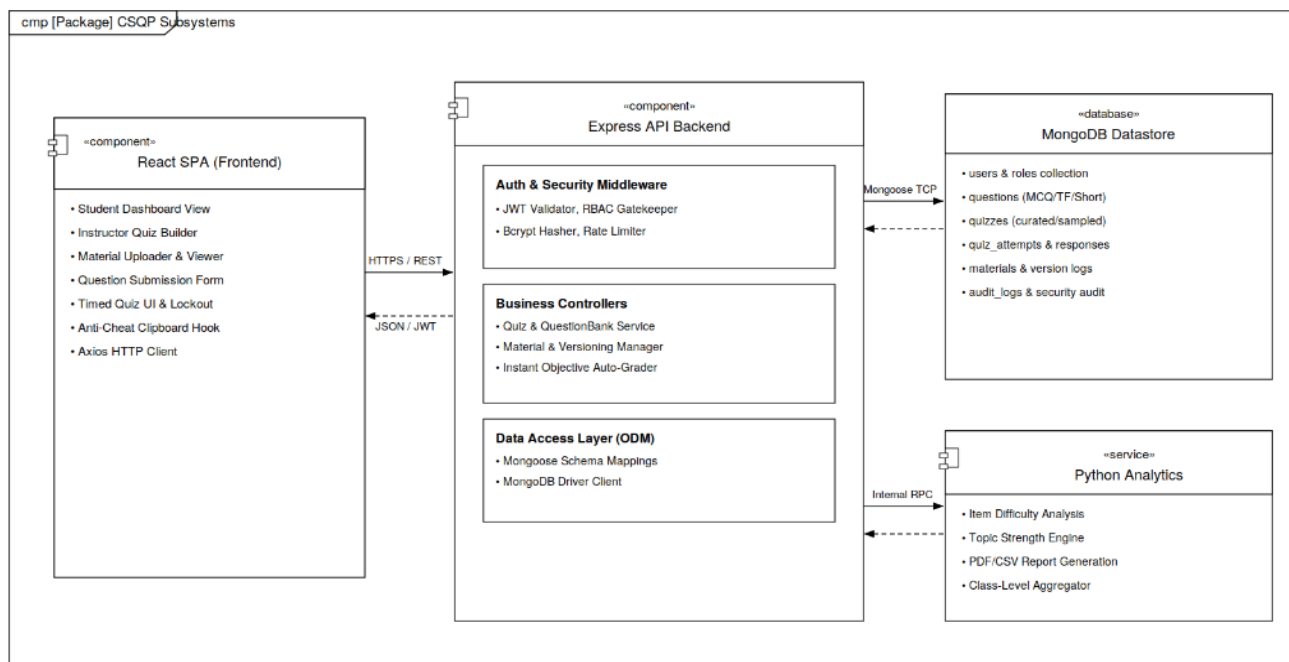


Figure 3.1: CSQP Subsystems Component Architecture Diagram

3.4 Component Descriptions:

- **React SPA (Client Layer):** Single-page interface rendering responsive dashboards for Students, Instructors, and Admins. Manages local client state, countdown timer clocks, input form validation, and clipboard event locks.
- **Express API Backend (Application Layer):** Central router exposing REST endpoints. Hosts security middleware (JWT authentication, rate limiting, RBAC gatekeepers), controllers for materials and quizzes, and auto-grading engines.
- **MongoDB Datastore (Persistence Layer):** Document database accessed through Mongoose schemas. Houses collections for user credentials, study notes metadata, question banks, quiz attempts, and audit trails.
- **Python Analytics (Worker Service):** Dedicated service executing heavy statistical queries, item difficulty distributions, student trend analysis, and PDF/CSV report generation without stalling the Node.js event loop.

3.5 Chosen Architecture Pattern and Rationale:

We chose a **3-Tier Layered Architecture with a Specialized Analytics Worker**. A pure monolithic approach would risk blocking the Node event loop during heavy student report generation and statistical aggregations. Conversely, an enterprise microservices mesh was rejected as unnecessarily complex for a semester academic project. The layered model with an internal worker cleanly decouples presentation, business rules, and analytical processing.

3.6 Technology Stack & Data Stores:

- **Frontend:** React.js (v18), Axios client, HTML5/CSS3 responsive layouts conforming to WCAG 2.1 AA.
- **Backend API:** Node.js (v18+), Express.js REST framework, Bcrypt hashing, JSON Web Tokens.
- **Database:** MongoDB (v6+) with Mongoose ODM indexing.
- **Analytics & Reports:** Python 3.10 with pandas and reportlab libraries.
- **Storage:** Local file storage / cloud bucket for notes up to 25MB with version tracking.

3.7 Risks & Mitigations:

- *Risk:* Database bottleneck during high concurrent quiz submissions. *Mitigation:* Compound indexes on student, quiz, and attempt status; lean JSON document writes.
- *Risk:* Credential brute-forcing. *Mitigation:* Express rate-limiting and automated 15-minute account lockouts following 5 consecutive failed logins.
- *Risk:* Quiz answer key leakage in browser dev tools. *Mitigation:* Correct answers stripped on server before payload transmission; objective items graded server-side upon submit.

3.8 Traceability to Requirements:

SRS Requirement ID	Architectural Component / Module	Design Element / Implementation
CSQP-F-001 to 004, SR-001 to 005	AuthModule & SecurityLayer	JWT Auth Middleware, Bcrypt Salting, Account Lockout Handler
CSQP-F-010 to 013	MaterialModule & StorageTier	Multipart Upload Controller, Text Search Indexes, Version Array
CSQP-F-020 to 024	QuestionBankModule & QuizModule	Moderation Controller, Quiz Sampling Engine, CSV Bulk Parser
CSQP-F-030 to 033	QuizAttemptModule & AutoGrading	Active Session Guard, Countdown Validator, Instant Grading Rule Engine
CSQP-F-040 to 043, NF-001	AnalyticsModule & Python Engine	Student/Class Aggregators, Item Difficulty Engine, PDF Exporter

3.9 Security Architecture & Threat Modeling (STRIDE)

The platform enforces multi-layered defense mapped against the STRIDE threat model:

Threat Category	Threat Scenario in CSQP	Architectural Mitigation Applied
Spoofing	Attacker impersonating a student or instructor to submit tests or manipulate grades.	Signed JWT access tokens (1h expiry), rotating HttpOnly refresh cookies, email OTP verification.
Tampering	Intercepting or modifying quiz answers, file uploads, or question difficulty ratings in transit.	Enforced TLS 1.2+ encryption for all traffic; server-side payload validation and checksum verification.
Repudiation	User denying taking a quiz or instructor denying question approval/rejection action.	Immutable audit log collection capturing user ID, IP address, action, and timestamp.
Information Disclosure	Leaking exam answer keys or exposing student personal academic records.	Correct answer keys stripped from quiz delivery payloads; passwords hashed via salted bcrypt.
Denial of Service (DoS)	Flooding login or quiz submission endpoints to crash server during active exam windows.	IP-level rate limiting on auth routes; 15-minute account lockout after 5 consecutive failed logins.
Elevation of Privilege	Student attempting to approve questions, create quizzes, or view class-wide analytics.	API-level RBAC route middleware verifying token role claims; unauthorized actions return HTTP 403.

4. Design

4.1 Design Overview: The system design decomposes functionality into cohesive controllers communicating with Mongoose ODM models. Business logic handles validation and anti-cheating state before updating persistent storage.

4.2 UML Sequence Diagrams:

Sequence Diagram 1: Timed Quiz Attempt, Anti-Cheating & Auto-Grading: Traces quiz startup, sanitization of answer keys, timer enforcement, and immediate objective scoring (< 2s):

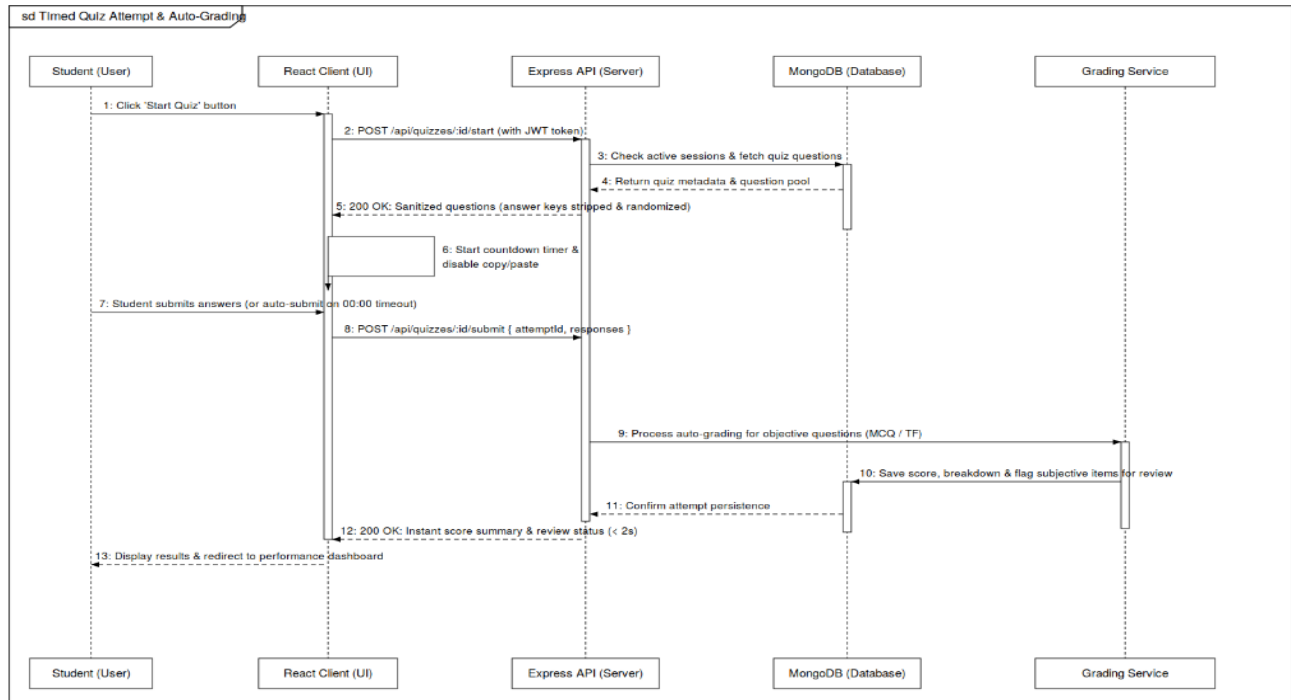


Figure 4.1: UML Sequence Diagram - Timed Quiz Attempt & Auto-Grading

Sequence Diagram 2: Question Contribution & Moderation Workflow: Illustrates peer question contribution where student items remain pending until reviewed and approved by an instructor:

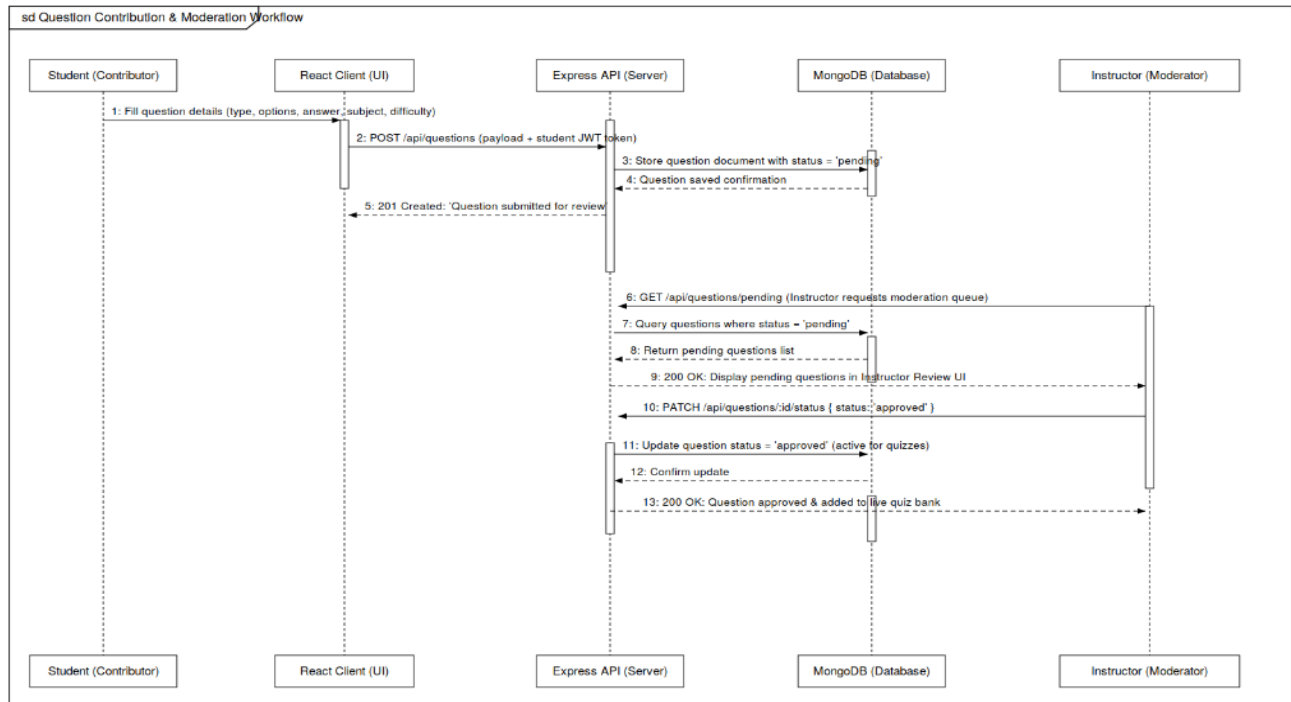


Figure 4.2: UML Sequence Diagram - Question Moderation & Approval Workflow

4.3 API Design:

Interface definitions for core subsystem components communicating over HTTPS with JSON payloads. Protected endpoints require Authorization: Bearer <JWT>:

Endpoint & Method	Role / Auth	Request Payload (DTO)	Response & Status	Error Codes
POST /api/auth/login	Public	{ email, password }	200 OK: { accessToken, user }	401 (Invalid creds) 429 (Account locked)
POST /api/materials/upload	Student	Multipart: file, title, subject, tags	201 Created: Material metadata	400 (File > 25MB) 401 (Unauthorized)
POST /api/questions	Student, Instructor	{ type, text, options, answer, subject, difficulty }	201 Created: { id, status }	400 (Validation fail) 401 (Unauthorized)
PATCH /api/questions/:id/status	Instructor	{ status: 'approved' 'rejected' }	200 OK: Updated question	403 (Forbidden: student) 404 (Not found)
POST /api/quizzes/:id/start	Student	{ } (Session check)	200 OK: Randomized questions (no keys)	409 (Active attempt exists) 404 (Quiz not found)
POST /api/quizzes/:id/submit	Student	{ attemptId, responses: [{ qId, answer }] }	200 OK: { score, isComplete }	400 (Malformed answers) 403 (Invalid attempt session)

4.4 Error Handling, Logging & Monitoring:

- **Standardized Error Envelope:** All controllers route failures to a centralized error middleware returning: { "success": false, "statusCode": 400, "error": "VALIDATION_ERROR", "message": "...", "timestamp": "..." }.
- **Logging Policy:** User passwords, credit tokens, and raw credentials are never logged. Operational events (auth failures, logout occurrences, quiz submissions) are written to local logs with request IDs.
- **Monitoring:** Health check route (GET /api/health) monitors MongoDB connection status, memory consumption, and quiz submission latency to ensure 99.5% availability.

4.5 UX Design:

- **Responsive Layout:** Adaptable grid for desktop and mobile viewports with accessible contrast conforming to WCAG 2.1 AA.
- **Quiz Interface:** Clear sticky countdown timer bar, prominent question progress indicator, and auto-submit alert modal.
- **Accessibility:** Complete keyboard navigation support (Tab/Enter/Space) across quiz forms and dashboard cards.

4.6 Open Issues & Next Steps:

- **Automated Subjective Grading:** Future integration of NLP models to evaluate short-answer responses automatically.
- **Collaborative Live Rooms:** Planned implementation of WebSockets for live group quizzes and peer study sessions.

5. Appendices

5.1 Glossary:

- **ADR:** Architecture Decision Record.
- **JWT:** JSON Web Token, used for stateless authentication.
- **MERN:** Technology stack comprising MongoDB, Express, React, and Node.js.
- **RBAC:** Role-Based Access Control enforcing role permissions.
- **STRIDE:** Threat modeling classification covering Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.
- **TLS:** Transport Layer Security protocol encrypting network traffic.

5.2 References:

1. IEEE Std 1016-2009, *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*.
2. IEEE Std 42010-2011, *Systems and Software Engineering — Architecture Description*.
3. OWASP Top 10 Web Application Security Risks (2021).
4. W3C Web Content Accessibility Guidelines (WCAG) 2.1 AA.

5.3 Tools:

- **Modeling & Diagramming:** draw.io, PlantUML.
- **API Testing & Documentation:** Postman, Swagger / OpenAPI 3.0.
- **Development & Version Control:** Visual Studio Code, Git, GitHub.